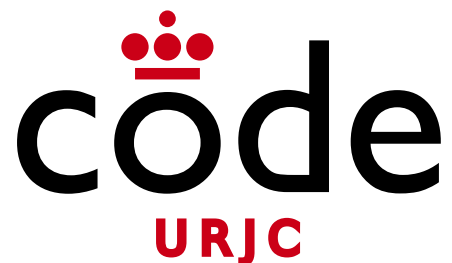


Desarrollo Web

Tema 4.9: Seguridad en Spring REST



©2026

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

APIs REST seguras

- Existen muchas **formas diferentes** de gestionar usuarios en una **API REST**
 - **Autenticación básica**
 - Autenticación con Tokens
 - OAuth1 y OAuth2

Autenticación básica

- La forma más sencilla de autenticar a un usuario que usa una API REST es incluir el **login** y la **password** en cada petición
- Para ello se usa el mecanismo conocido como **Autenticación de acceso básica** (*basic access authentication* o *basic auth*) de http

https://en.wikipedia.org/wiki/Basic_access_authentication

Autenticación básica

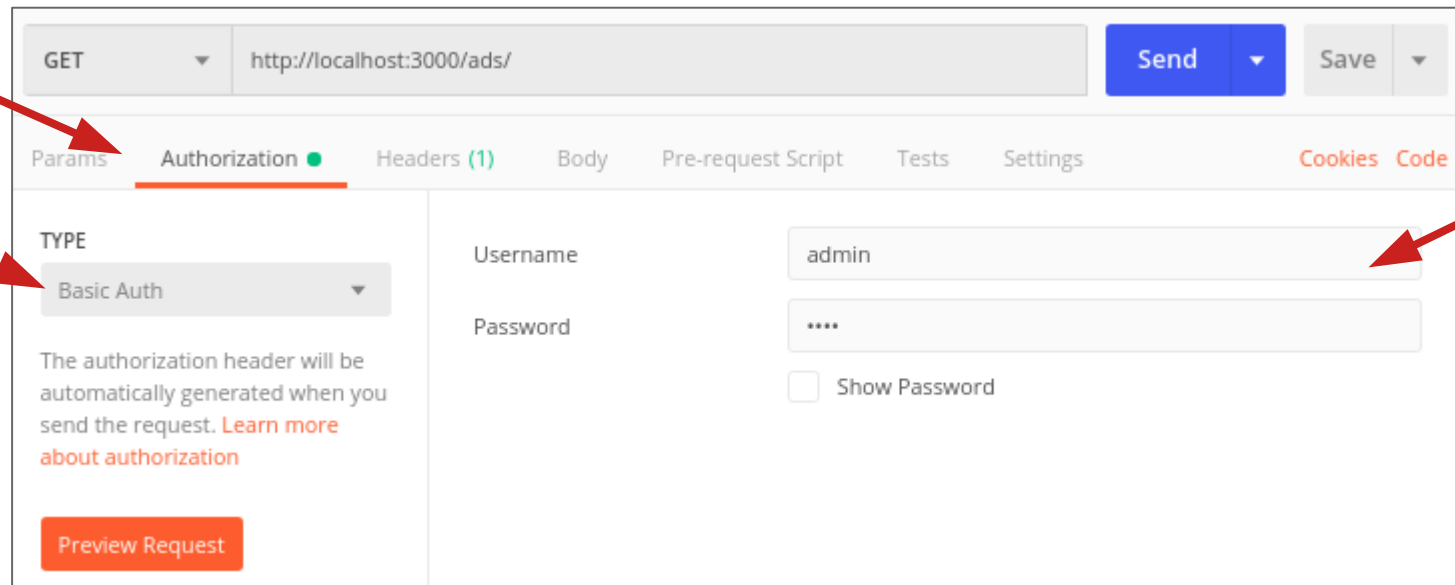
- Las credenciales (login y password) se envían en la cabecera **Authorization**
- El valor se calcula como:
`'Basic '+base64(login+' : '+password)`
- Ejemplo para login “Aladdin” y password “OpenSesame”

Authorization: Basic QWxhZGRpbjppPcGVuU2VzYW11

Autenticación básica

rest_ejem15_security_basic

- En Postman se especifica el tipo de Autorización “**Basic Auth**”, el login y la password y él genera la cabecera



The screenshot shows the Postman interface with the 'Authorization' tab selected. The request method is 'GET' and the URL is 'http://localhost:3000/ads/'. The 'Authorization' tab is active, showing the 'Basic Auth' type selected. The 'Username' field is filled with 'admin' and the 'Password' field is masked with '****'. A 'Show Password' checkbox is present and unchecked. A 'Preview Request' button is at the bottom left of the authorization section. Red arrows point to the 'Basic Auth' dropdown, the 'Username' field, and the 'Password' field.

GET http://localhost:3000/ads/ Send Save

Params **Authorization** Headers (1) Body Pre-request Script Tests Settings Cookies Code

TYPE

Basic Auth

The authorization header will be automatically generated when you send the request. [Learn more about authorization](#)

Preview Request

Username admin

Password ****

☐ Show Password

Autenticación básica

rest_ejem15_security_basic

```
@Configuration
@EnableWebSecurity
public class RestSecurityConfig {

    ...

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        http.authenticationProvider(authenticationProvider());

        http.authorizeHttpRequests(authorize -> authorize
            // PRIVATE ENDPOINTS
            .requestMatchers(HttpMethod.POST, "/api/books/**").hasRole("USER")
            .requestMatchers(HttpMethod.PUT, "/api/books/**").hasRole("USER")
            .requestMatchers(HttpMethod.DELETE, "/api/books/**").hasRole("ADMIN")
            // PUBLIC ENDPOINTS
            .anyRequest().permitAll()
        );

        // Disable Form login Authentication
        http.formLogin(formLogin -> formLogin.disable());

        // Disable CSRF protection (it is difficult to implement in REST APIs)
        http.csrf(csrf -> csrf.disable());

        // Enable Basic Authentication
        http.httpBasic(Customizer.withDefaults());

        // Stateless session
        http.sessionManagement(management -> management.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        return http.build();
    }
}
```

APIs REST seguras

- Existen muchas **formas diferentes** de gestionar usuarios en una **API REST**
 - Autenticación básica
 - **Autenticación con Tokens**
 - OAuth1 y OAuth2

Autenticación con tokens

- La autenticación **enviando la contraseña** en cada petición **no** es recomendable
- Si un atacante llegase a **interceptar una única petición**, se podría hacer pasar por el usuario
- Es recomendable usar la contraseña una única vez para obtener un **token** con el que poder hacer las siguientes peticiones
- Ese token suele tener **caducidad**, limitando el impacto en caso de ataque

Autenticación con tokens

- **Paso 1) Se hace una petición con username y contraseña para obtener el token**

Se puede usar Basic Auth para autenticar o enviar las credenciales en el body

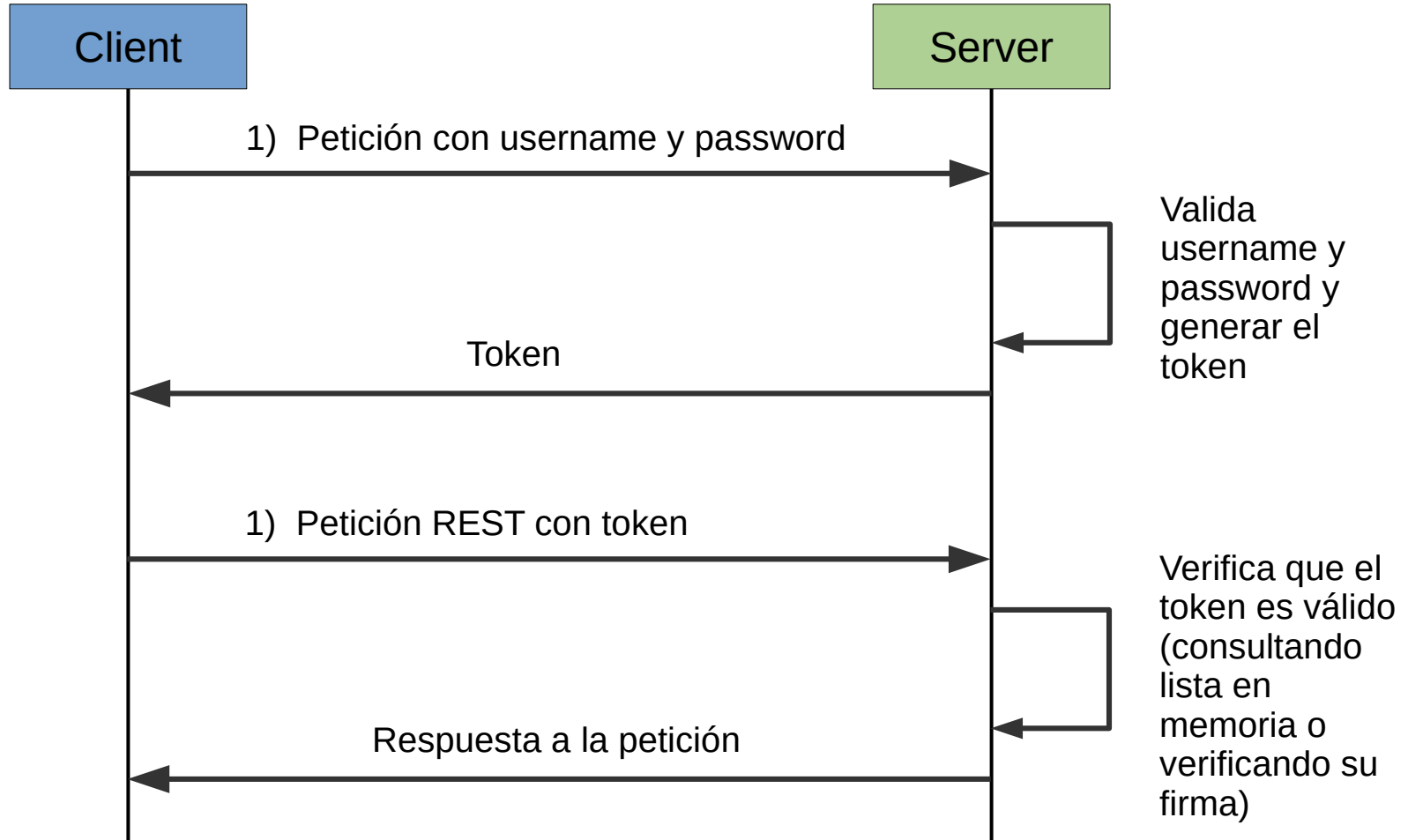
Se genera un token y se devuelve

- **Paso 2) Se hace una petición con el token**

Se verifica si el token es válido

Si lo es, se atiende la petición

Autenticación con tokens



Autenticación con tokens

- **Tipos de tokens**

Número aleatorio grande difícil de adivinar

- Fácil de generar
- Se mantiene una lista de tokens válidos asociados al nombre de cada usuario
- Es el funcionamiento de las cookies de sesión (JSESSIONID)

Autenticación con tokens

- Tipos de tokens

Número aleatorio grande difícil de adivinar

- En sistemas distribuidos la gestión de la lista de tokens puede limitar la escalabilidad
- Lo ideal es que el propio token tuviese todo lo necesario para saber si es válido o no

Autenticación con tokens

- Tipos de tokens

JSON firmado y con tiempo de expiración

- El token puede llevar información como **username**, rol, permisos
- Como el token está **firmado**, su contenido no se puede alterar (o se detectaría)
- **Elimina** la necesidad de mantener una lista de tokens válidos y usernames
- El token se **invalida** cuando caduca

Autenticación con tokens

- JSON Web Tokens



JSON Web Tokens are an open, industry standard [RFC 7519](https://tools.ietf.org/html/rfc7519) method for representing claims securely between two parties.

<https://jwt.io/>

JWT (JSON Web Tokens)

- Los tokens JWT siempre van **firmados** (para que se pueda **verificar su autenticidad**)
- Puede **cifrarse o no** (no suele haber mucho que ocultar)
- Pueden usarse de muchas formas diferentes (son muy **flexibles**)
- Un **servidor** puede **generar** un token y **verificarse** en otro **servidor** (ideal para sistemas **distribuidos**)

JWT (JSON Web Tokens)

- ¿Cómo se transmiten los tokens?
- Cabeceras http
 - El cliente tiene que recoger la cabecera de la operación de login
 - Programar el envío en el resto de peticiones
 - Para aplicaciones SPA requiere manipular el token en código JavaScript
 - Si se quiere soportar el refresco de la página se tiene que guardar en el localStorage

JWT (JSON Web Tokens)

- ¿Cómo se transmiten los tokens?
- **Cookies**
 - Cuando el cliente es un browser, el proceso de recogida y envío de la cookie se hace de forma automática
 - El token no se puede manipular desde JavaScript porque la cookie es **HttpOnly**
 - No se guarda token en **local storage**
 - La opción más **segura para aplicaciones SPA**

JWT (JSON Web Tokens)

rest_ejem16_security_jwt

- Implementación en Spring
- Se usa una librería externa para gestión de Token JWT
- Requiere un procesamiento manual de los tokens y de las cookies

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>${jjwt.version}</version>
</dependency>

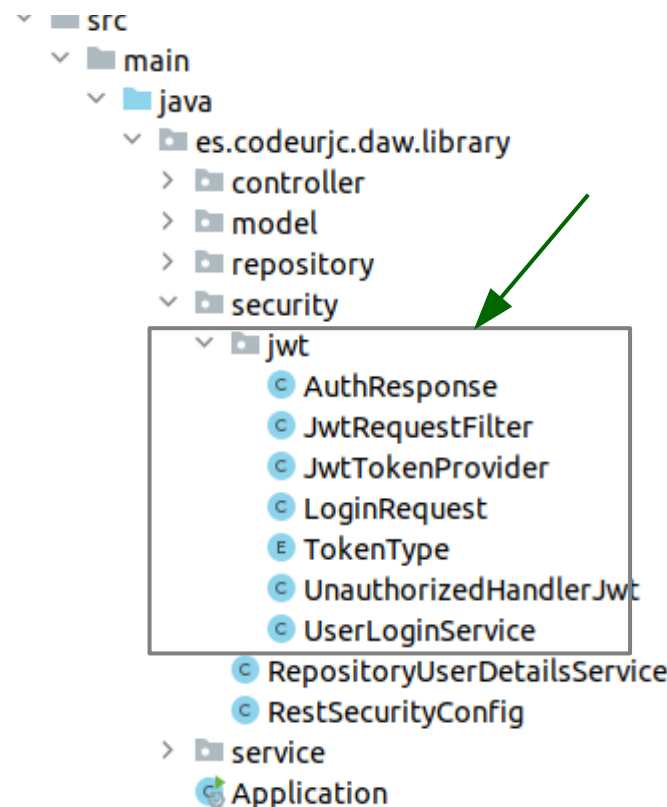
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>${jjwt.version}</version>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</
artifactId>
  <version>${jjwt.version}</version>
  <scope>runtime</scope>
</dependency>
```

JWT (JSON Web Tokens)

rest_ejem16_security_jwt

- Implementación en Spring
- Se proporciona código de gestión de tokens que se usa tal cual (no necesita ser adaptado para cada aplicación)



JWT (JSON Web Tokens)

rest_ejem16_security_jwt

- Implementación en Spring
- Dentro de este paquete, en la clase **TokenType** puede configurarse el tiempo de vida que tendrán los tokens

```
public enum TokenType {
    ACCESS(Duration.ofMinutes(5), "AuthToken"),
    REFRESH(Duration.ofDays(7), "RefreshToken");
    ...
}
```

JWT (JSON Web Tokens)

rest_ejem16_security_jwt

- Endpoints REST para login

```
@RestController
@RequestMapping("/api/auth")
public class LoginController {

    @Autowired
    private UserLoginService userService;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(
        @RequestBody LoginRequest loginRequest,
        HttpServletResponse response) {

        return userService.login(response, loginRequest);
    }

    @PostMapping("/refresh")
    public ResponseEntity<AuthResponse> refreshToken(
        @CookieValue(name = "RefreshToken", required = false) String refreshToken,
        HttpServletResponse response) {

        return userService.refresh(response, refreshToken);
    }

    @PostMapping("/logout")
    public ResponseEntity<AuthResponse> logOut(HttpServletResponse response) {
        return ResponseEntity.ok(new AuthResponse(Status.SUCCESS, userService.logout(response)));
    }
}
```

Se obtienen las cookies y las credenciales del cliente y se envían al UserLoginService

JWT (JSON Web Tokens)

rest_ejem16_security_jwt

- Cambios al SecurityConfig

```
@Configuration
@EnableWebSecurity
public class RestSecurityConfig {

    ...

    //Expose AuthenticationManager as a Bean to be used in other services

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration
authConfig) throws Exception {
        return authConfig.getAuthenticationManager();
    }

    ...
}
```

Es necesario publicar como un Bean el AuthenticationManager para que el código de gestión de tokens JWT lo pueda usar

JWT (JSON Web Tokens)

rest_ejem16_security_jwt

```
@Configuration
@EnableWebSecurity
public class RestSecurityConfig {

    @Autowired
    private JwtRequestFilter jwtRequestFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        ...

        // Disable Form login Authentication
        http.formLogin(formLogin -> formLogin.disable());

        // Disable CSRF protection (it is difficult to implement in REST APIs)
        http.csrf(csrf -> csrf.disable());

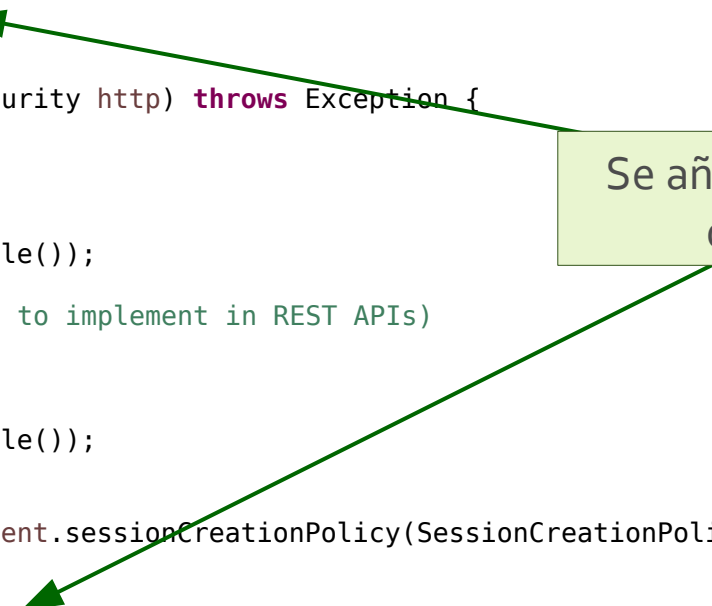
        // Disable Basic Authentication
        http.httpBasic(httpBasic -> httpBasic.disable());

        // Stateless session
        http.sessionManagement(management -> management.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        // Add JWT Token filter
        http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Se añade un filtro de JWT



JWT (JSON Web Tokens)

rest_ejem16_security_jwt

```
@Configuration
@EnableWebSecurity
public class RestSecurityConfig {

    @Autowired
    private JwtRequestFilter jwtRequestFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        ...

        // Disable Form login Authentication
        http.formLogin(formLogin -> formLogin.disable());

        // Disable CSRF protection (it is difficult to implement in REST APIs)
        http.csrf(csrf -> csrf.disable());

        // Disable Basic Authentication
        http.httpBasic(httpBasic -> httpBasic.disable());

        // Stateless session
        http.sessionManagement(management -> management.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        // Add JWT Token filter
        http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Se deshabilita tanto basic auth como form login

JWT (JSON Web Tokens)

rest_ejem16_security_jwt

```
@Configuration
@EnableWebSecurity
public class RestSecurityConfig {

    @Autowired
    private JwtRequestFilter jwtRequestFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        ...

        // Disable Form login Authentication
        http.formLogin(formLogin -> formLogin.disable());

        // Disable CSRF protection (it is difficult to implement in REST APIs)
        http.csrf(csrf -> csrf.disable());

        // Disable Basic Authentication
        http.httpBasic(httpBasic -> httpBasic.disable());

        // Stateless session
        http.sessionManagement(management -> management.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        // Add JWT Token filter
        http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Se desactiva la
gestión de sesión
(cada petición lleva
credenciales)



APIs REST seguras

- Existen muchas **formas diferentes** de gestionar usuarios en una **API REST**
 - Autenticación básica
 - Autenticación con Tokens
 - **OAuth1 y OAuth2**

OAuth1 y OAuth2

- OAuth1 y OAuth2 son estándares de seguridad que permiten a un usuario **conceder permisos** a una aplicación para que acceda a **sus datos** en **otra aplicación**
- Por ejemplo, permitir que una web acceda a tus datos de **Facebook**

OAuth1 y OAuth2

- Existen diferentes mecanismos para conseguir estos objetivos llamados **flujos** (*flows*)
- Ejemplo de un flujo: Supongamos que una aplicación “**Terrible pun of the day**” quiere enviar un chiste a los **contactos de tu agenda** del proveedor de GMail.

<https://developer.okta.com/blog/2019/10/21/illustrated-guide-to-oauth-and-oidc>



ALREADY
LOGGED IN?

you'RE THE BEST FRIEND EVER!



OAuth1 y OAuth2

- **No lo vamos a ver en detalle** porque no es sencillo de implementar
- Los servicios de login social con **Google, Facebook, Twitter (X), GitHub...** se basan en estos estándares pero tienen ciertas particularidades
- Para implementar login social con alguno de estos servicios hay que revisar la **documentación de cada uno de ellos**